Formal Verification

BY: Eng Ali Eltemsah

© 2022 Eng. Ali Eltemsah

CONTENTS

- Introduction to Formal Verification
- Formal Verification Components
- Design Equivalence Checking
- Formality Flow
  - Formality Setup
  - Load Guidance
  - Load Reference Design
  - Load Implementation Design
  - Load Setup File
  - Match
  - Verify
  - Debug
  - Report

© 2022 Eng. Ali Eltemsah

Introduction to Formal Verification

- Formal Verification refers to the process of establishing functional equivalence of RTL model which is called the reference design(ref) and a corresponding netlist which is called the implemented design (Impl) without running simulations. Formal Verification makes sure that no logic change is happened.
  - The corresponding netlist can be the netlist post-synthesis, netlist post-DFT or netlist post-PNR.

- Formal verification is the overarching term for a collection of techniques that use static analysis based on mathematical transformations and Boolean functions to determine the correctness of hardware.

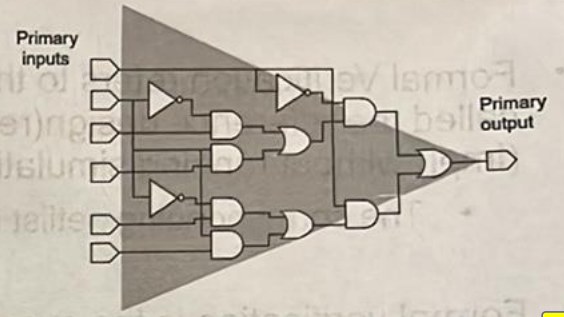
- Why Formal Verification?
  - As the ASICs are becoming larger and complex, the netlist simulation is becoming a very time consuming process, and its not worth running weeks of netlist simulations corresponding to a very small change in RTL just to establish that the netlist is good. Formal verification is an answer to such a problem.
  - Formal verification is also a double check on your synthesis tool, that it is doing the right job.

- Examples of EDA tools for formal verification.
  1. Formality from Synopsys
  2. Co-formal from Cadence
  3. Formal-pro from Mentor

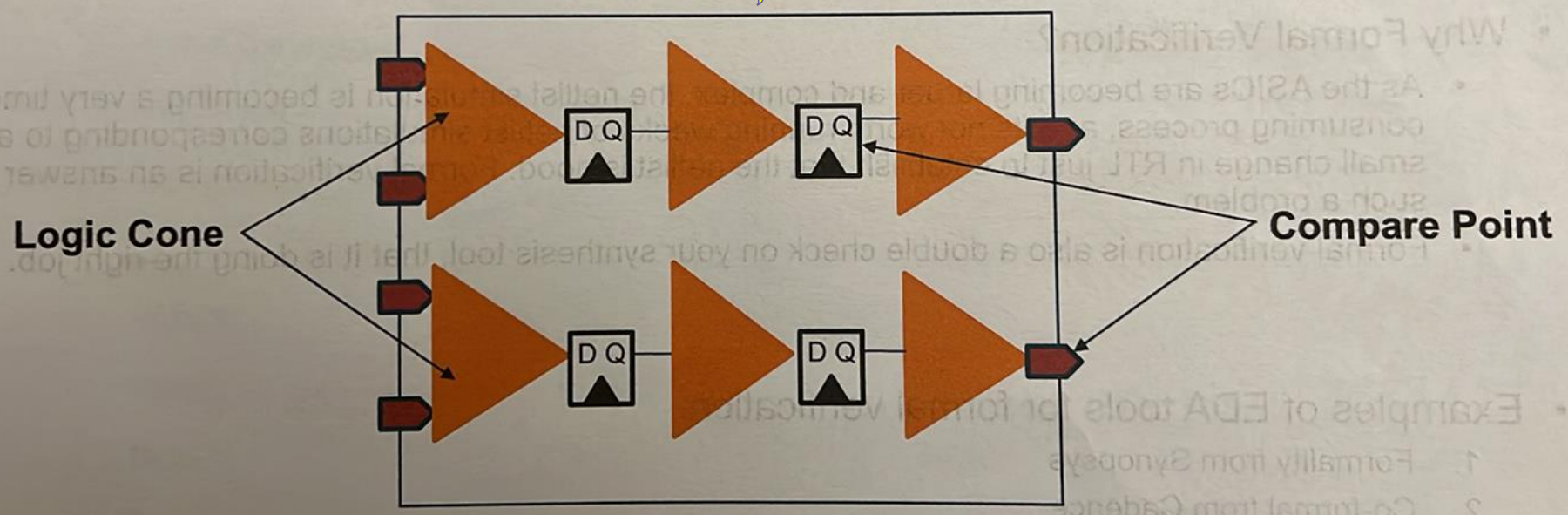
© 2022 Eng. Ali Eltemsah

Formal Verification Components

1. **Logic Cone**
   - It consists of combinational logic that drive a compare point



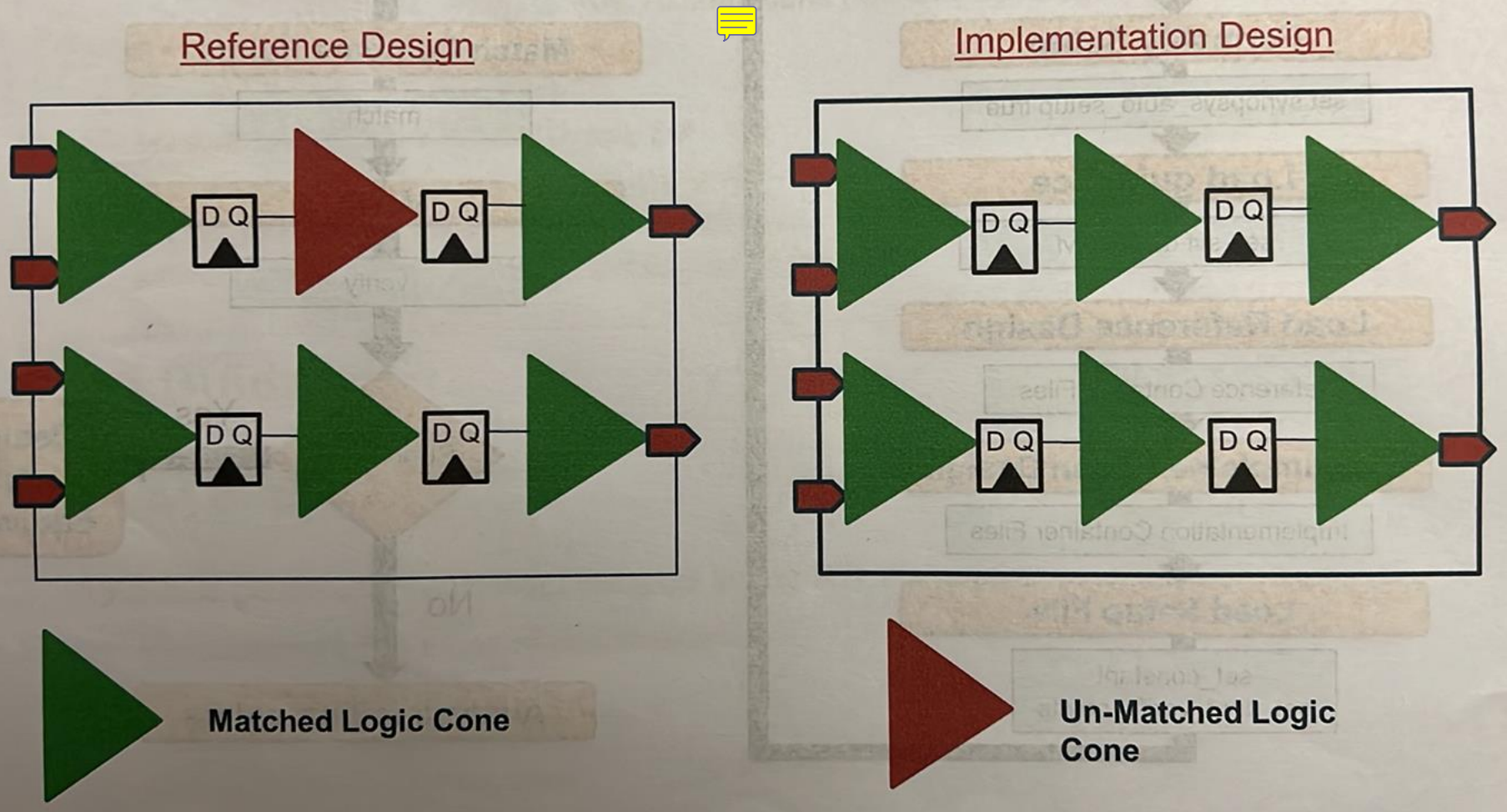
2. **Compare Point**
   - A compare point is a design object used as a combinational logic endpoint during verification
   - A compare point can be a primary output, register, latch, black box input pin



© 2022 Eng. Ali Eltemsah

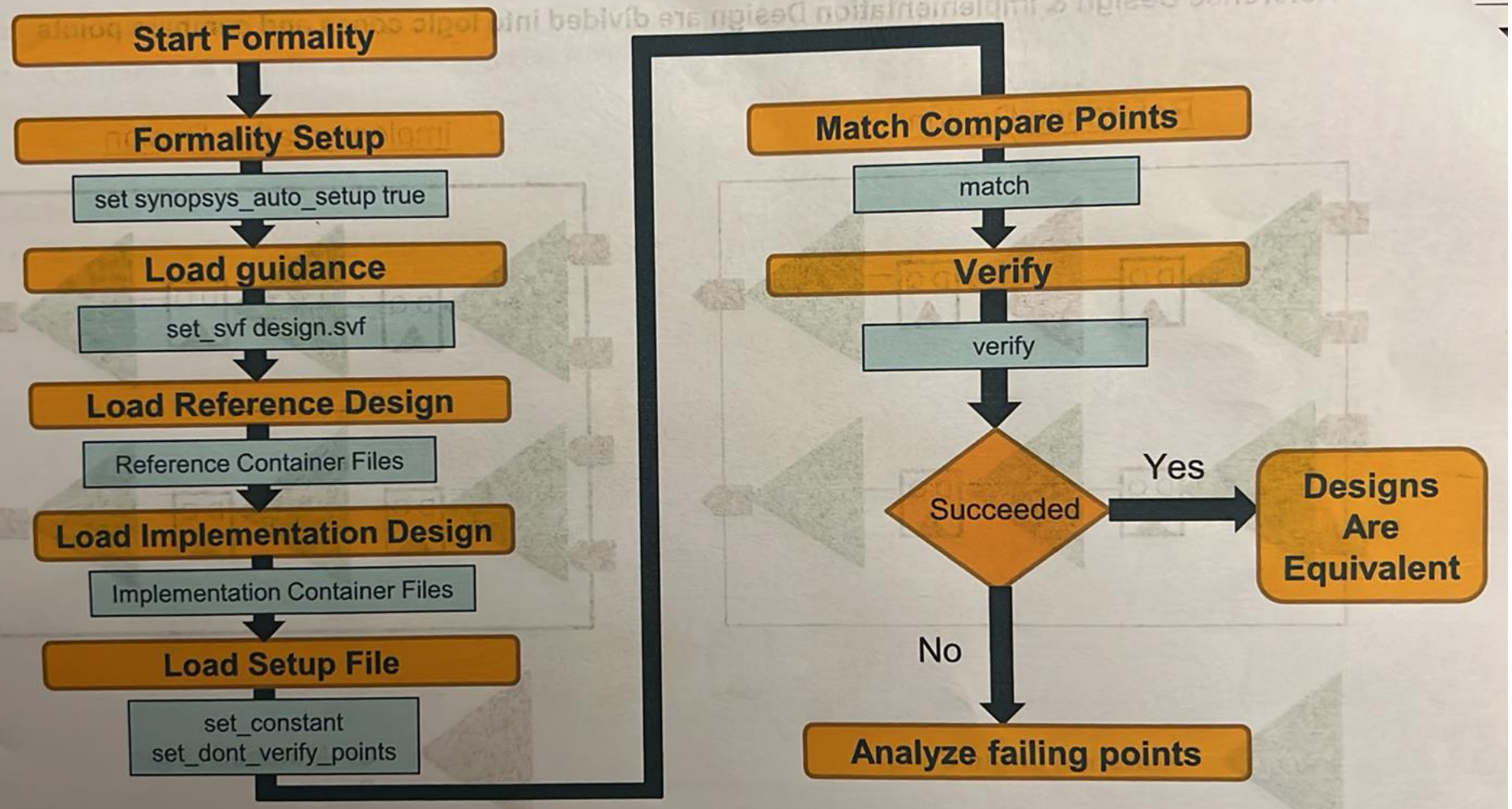
Design Equivalence Checking

- Reference Design & Implementation Design are divided into logic cones and compare points



© 2022 Eng. Ali Eltemsah

Formality Flow



© 2022 Eng. Ali Eltemsah

Formality Setup

1) set synopsys_auto_setup true

- When set synopsys_auto_setup to true. Assumptions made in Synthesis will also be made in formality
  - set hdlin_ignore_parallel_case false
  - set hdlin_ignore_full_case false
  - set verification_set_undriven_signals 0:X
  - set hdlin_error_on_mismatch_message false
  - set svf_ignore_unqualified_fsm_information false
  - set verification_verify_directly_undriven_output false

Load Guidance

1) set_svf "design.svf"

- SVF File is generated from design compiler during RTL synthesis.
- SVF File contains these types of information:
  1. Object name changes
  2. Constant register optimizations
  3. Duplicate and merged registers

© 2022 Eng. Ali Eltemsah

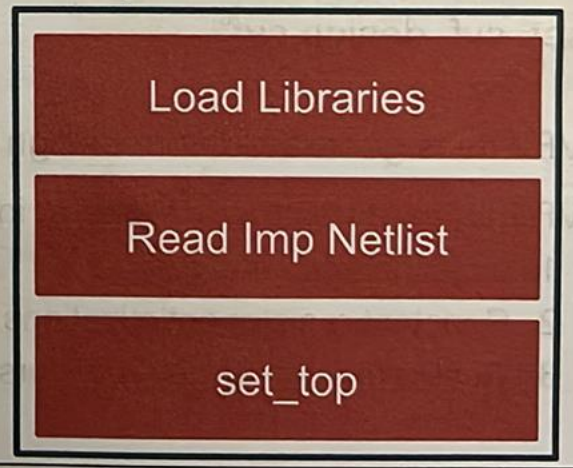
Load Reference & Implementation Design

1) Load Libraries
   - read_db -container Ref [list $SVTDB $HVTDB]
2) Read Design Files
   - read_verilog -container Ref "fcs.v"
   - read_verilog -container Ref "regfile.v"
   - read_verilog -container Ref "serializer.v"
   - read_verilog -container Ref "system_top.v"
3) set_top design
   - set_reference_design system_top
   - set_top system_top

Reference Container

1) Load Libraries
   - read_db -container Imp [list $SVTDB $HVTDB]
2) Read Design Files
   - read_ddc -container Imp "system_top.ddc"
3) set_top design
   - set_implementation_design system_top
   - set_top system_top

Implementation Container



© 2022 Eng. Ali Eltemsah

Load Setup Commands

1. set_dont_verify_points
   - This command is used to exclude some compare points from verification
   - Example:-
     - `set_dont_verify_points -type port Ref:/WORK/*/SO`
     - `set_dont_verify_points -type port Imp:/WORK/*/SO`

don't verify scan out ports as a compare point as it is existed only after dft insertion and not existed in the RTL

2. set_constant
   - This command is used to tie some ports or pins to constant value whether 0 or 1
   - Example:-
     - `set_constant Ref:/WORK/*/test_mode 0`
     - `set_constant Imp:/WORK/*/test_mode 0`

```
set_constant Ref:/WORK/*/SE 0
set_constant Imp:/WORK/*/SE 0
```

3. set verification_set_undriven_signals X
   - This command to allow the formality to treat undriven nets and pins as X during verification

© 2022 Eng. Ali Eltemsah

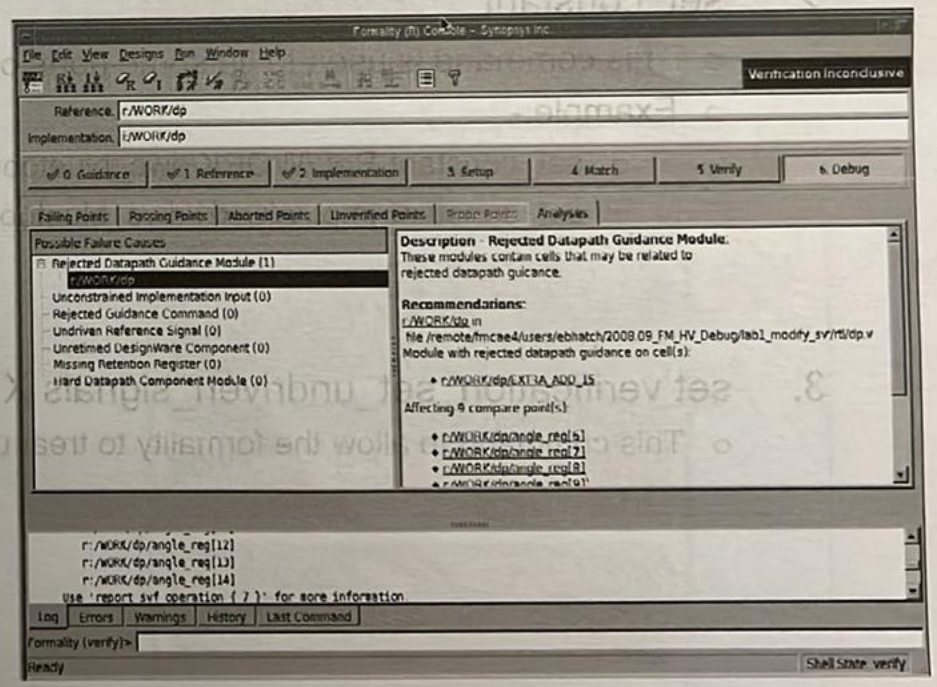
match | Verify | Debug | Report

1. match
   - It is used to match the compare points

```
Status:  Matching...

********************************** Matching Results ************************
 32 Compare points matched by name
 0 Compare points matched by signature analysis
 0 Compare points matched by topology
 96 Matched primary inputs, black-box outputs
 0(0) Unmatched reference(implementation) compare points
 0(0) Unmatched reference(implementation) primary inputs, black-box outputs
 15(0) Unmatched reference(implementation) unread points
****************************************************************************

Status:  Verifying...

Status:  Matching hierarchy...
```

2. verify
   - It is used to verify the equivalence of the compare points

3. Debug through Formality GUI



4. Reporting
   - report_passing_points > "passing_points.rpt"
   - report_failing_points > "failing_points.rpt"
   - report_aborted_points > "aborted_points.rpt"
   - report_unverified_points > "unverified_points.rpt"

© 2022 Eng. Ali Eltemsah